Taller de repaso: teoría de aprendizaje de máquinas

Juan Diego Restrepo Hernández

9 de noviembre de 2026

Punto 1.1: Frontera de decisión y clasificación de puntos

1. Ecuación de la frontera de decisión La frontera de decisión corresponde a los puntos $x \in \mathbb{R}^2$ para los cuales $w^\top x + b = 0$. Reemplazando $w = [1, 2]^\top$ y $b = -3$:

$$1 \cdot x_1 + 2 \cdot x_2 - 3 = 0 \implies x_1 + 2x_2 - 3 = 0.$$

Expresando x_2 en función de x_1:

$$x_2 = -\frac{1}{2}x_1 + \frac{3}{2}.$$

2. Evaluación de las observaciones Utilizando la regla de predicción:

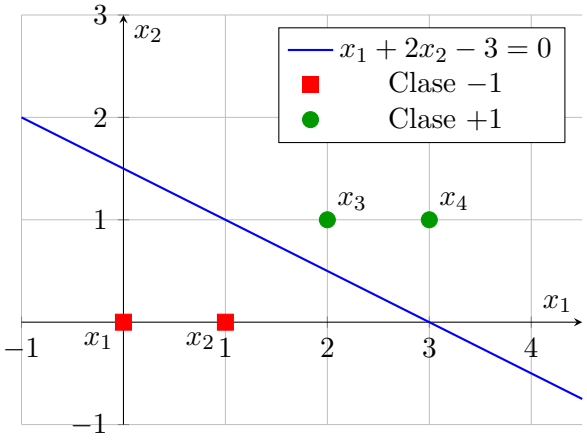
$$\hat{y} = \begin{cases} +1, & w^\top x + b > 0, \\ -1, & w^\top x + b \leq 0. \end{cases}$$

- Para $x_1 = (0, 0)^\top$: $w^\top x_1 + b = 1(0) + 2(0) - 3 = -3 \leq 0 \implies \hat{y}_1 = -1$.
- Para $x_2 = (1, 0)^\top$: $w^\top x_2 + b = 1(1) + 2(0) - 3 = -2 \leq 0 \implies \hat{y}_2 = -1$.
- Para $x_3 = (2, 1)^\top$: $w^\top x_3 + b = 1(2) + 2(1) - 3 = 1 > 0 \implies \hat{y}_3 = +1$.
- Para $x_4 = (3, 1)^\top$: $w^\top x_4 + b = 1(3) + 2(1) - 3 = 2 > 0 \implies \hat{y}_4 = +1$.

Observación	(x_1, x_2)	$w^\top x + b$	**Condición**	**Clase asignada** ($\hat{y}$)
x_1	$(0, 0)$	-3	≤ 0	-1
x_2	$(1, 0)$	-2	≤ 0	-1
x_3	$(2, 1)$	$+1$	> 0	$+1$
x_4	$(3, 1)$	$+2$	> 0	$+1$

Cuadro 1: Resultados de clasificación para las observaciones dadas.

3. Gráfica de la frontera de decisión



Punto 1.2: Actualización del perceptrón y demostración del margen

1. Definición del problema Dado un ejemplo (x_i, y_i) con $x_i \in \mathbb{R}^2$ y $y_i \in \{-1, +1\}$, la regla de actualización del perceptrón ante una clasificación errónea es:

$$w_{\text{nuevo}} = w + \eta y_i x_i \tag{1}$$

$$b_{\text{nuevo}} = b + \eta y_i \tag{2}$$

Donde $\eta > 0$ representa la tasa de aprendizaje.

El margen funcional de la observación x_i con los parámetros actuales se define como:

$$m_i = y_i \left(w^\top x_i + b \right). \tag{3}$$

Se desea demostrar que, inmediatamente después de aplicar la regla de actualización sobre esta misma observación, el nuevo margen satisface:

$$m_{i,\text{nuevo}} = m_i + \eta \left(\|x_i\|_2^2 + 1 \right). \tag{4}$$

2. Demostración matemática paso a paso

1. **Expresión del nuevo margen:** Por definición, el nuevo margen $m_{i,\text{nuevo}}$ tras actualizar los parámetros w y b es:
$$m_{i,\text{nuevo}} = y_i \left(w_{\text{nuevo}}^\top x_i + b_{\text{nuevo}} \right).$$
2. **Sustitución de las reglas de actualización:** Reemplazando las Ecuaciones (1) y (2) en la expresión del nuevo margen:
$$m_{i,\text{nuevo}} = y_i \left[(w + \eta y_i x_i)^\top x_i + (b + \eta y_i) \right].$$

3. **Expansión del producto traspuesto:** Aplicando la propiedad distributiva del producto escalar $(A+B)^\top C = A^\top C + B^\top C$:

$$(w + \eta y_i x_i)^\top x_i = w^\top x_i + (\eta y_i x_i)^\top x_i = w^\top x_i + \eta y_i (x_i^\top x_i).$$

Recordando que la norma euclidiana al cuadrado se define como $\|x_i\|_2^2 = x_i^\top x_i$:

$$(w + \eta y_i x_i)^\top x_i = w^\top x_i + \eta y_i \|x_i\|_2^2.$$

4. **Reagrupamiento dentro del paréntesis:** Sustituyendo el resultado del paso anterior en la ecuación del margen:

$$\begin{aligned} m_{i,\text{nuevo}} &= y_i \left[w^\top x_i + \eta y_i \|x_i\|_2^2 + b + \eta y_i \right] \\ &= y_i \left[(w^\top x_i + b) + \eta y_i \|x_i\|_2^2 + \eta y_i \right]. \end{aligned}$$

Factorizando el término común ηy_i:

$$m_{i,\text{nuevo}} = y_i \left[(w^\top x_i + b) + \eta y_i \left(\|x_i\|_2^2 + 1 \right) \right].$$

5. **Distribución de y_i y simplificación:** Multiplicando y_i a cada término de la suma:

$$m_{i,\text{nuevo}} = \underbrace{y_i (w^\top x_i + b)}_{m_i} + \eta y_i^2 \left(\|x_i\|_2^2 + 1 \right).$$

Dado que $y_i \in \{-1, +1\}$, se cumple que $y_i^2 = 1$. Por lo tanto:

$$m_{i,\text{nuevo}} = m_i + \eta \left(\|x_i\|_2^2 + 1 \right).$$

$$\boxed{m_{i,\text{nuevo}} = m_i + \eta \left(\|x_i\|_2^2 + 1 \right)} \quad \blacksquare$$

3. Interpretación del resultado

- **Incremento estrictamente positivo:** Dado que la norma al cuadrado $\|x_i\|_2^2 \geq 0$ y la tasa de aprendizaje $\eta > 0$, el término de incremento es estrictamente positivo:

$$\Delta m = \eta \left(\|x_i\|_2^2 + 1 \right) \geq \eta > 0.$$

- **Corrección del error:** Si la observación x_i estaba mal clasificada, su margen inicial era no positivo ($m_i \leq 0$). La actualización garantiza sumar una cantidad positiva a su margen, desplazando la predicción del modelo en la dirección correcta (hacia un margen positivo $m_{i,\text{nuevo}} > m_i$).

- **Efecto de la magnitud de x_i:** El incremento del margen es proporcional a la magnitud vectorial $\|x_i\|_2^2$. Las observaciones más alejadas del origen ejercen un impacto mayor en el ajuste de la frontera de decisión durante un paso de actualización.

Punto 1.3: Ejercicio numérico de actualización del perceptrón

1. Datos iniciales Se consideran los siguientes datos del problema:

$$w = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad b = 0, \quad \eta = 0{,}5,$$
$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad y = +1.$$

2. Determinación de la necesidad de actualización Evaluamos la regla de predicción dada para la observación x:

$$f(x) = w^\top x + b = \begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0 = 0(2) + 0(1) + 0 = 0.$$

Aplicando la regla de decisión:

$$\hat{y} = \begin{cases} +1, & w^\top x + b > 0, \\ -1, & w^\top x + b \leq 0. \end{cases}$$

Dado que $f(x) = 0 \leq 0$, el modelo predice $\hat{y} = -1$.

Como la etiqueta real es $y = +1$ y la predicha es $\hat{y} = -1$, la observación está **mal clasificada** ($\hat{y} \neq y$). Por lo tanto, **sí corresponde actualizar los parámetros**.

3. Cálculo del margen inicial El margen original m antes de la actualización es:

$$m = y(w^\top x + b) = (+1)(0) = 0.$$

4. Actualización de los parámetros Aplicando las reglas de actualización del perceptrón con $\eta = 0{,}5$ y $y = +1$:

- **Nuevo vector de pesos (w_{nuevo}):**

$$\begin{aligned} w_{\text{nuevo}} &= w + \eta y x \\ &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} + (0{,}5)(+1) \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1{,}0 \\ 0{,}5 \end{bmatrix} = \begin{bmatrix} 1{,}0 \\ 0{,}5 \end{bmatrix}. \end{aligned}$$

- **Nuevo sesgo (b_{nuevo}):**

$$b_{\text{nuevo}} = b + \eta y = 0 + (0{,}5)(+1) = 0{,}5.$$

5. Cálculo del nuevo margen Calculamos el nuevo margen por dos métodos equivalentes para verificar el procedimiento:

1. **Cálculo directo por definición:**

$$\begin{aligned} m_{\text{nuevo}} &= y\left(w_{\text{nuevo}}^{\top} x + b_{\text{nuevo}}\right) \\ &= (+1)\left(\begin{bmatrix}1{,}0 & 0{,}5\end{bmatrix}\begin{bmatrix}2\\1\end{bmatrix} + 0{,}5\right) \\ &= 1{,}0(2) + 0{,}5(1) + 0{,}5 \\ &= 2{,}0 + 0{,}5 + 0{,}5 = 3{,}0. \end{aligned}$$

2. **Verificación mediante la fórmula del punto 1.2:** Calculamos la norma al cuadrado $\|x\|_2^2$:

$$\|x\|_2^2 = x_1^2 + x_2^2 = 2^2 + 1^2 = 4 + 1 = 5.$$

   Sustituyendo en $m_{\text{nuevo}} = m + \eta\left(\|x\|_2^2 + 1\right)$:

$$m_{\text{nuevo}} = 0 + 0{,}5(5 + 1) = 0{,}5(6) = 3{,}0.$$

Ambos métodos coinciden exactamente.

Resumen de resultados

- **¿Corresponde actualizar?:** Sí ($\hat{y} = -1 \neq y = +1$).
- **Nuevo vector de pesos:** $w_{\text{nuevo}} = [1{,}0, 0{,}5]^{\top}$.
- **Nuevo sesgo:** $b_{\text{nuevo}} = 0{,}5$.
- **Nuevo margen:** $m_{\text{nuevo}} = 3{,}0$.

Punto 2.1: Cálculo manual de k-NN

1. Definición y formulación Dada la observación de consulta $q = (1, 1)^{\top}$ y la métrica de distancia euclidiana en $\mathbb{R}^2$:

$$d(q, x_i) = \sqrt{(q_1 - x_{i,1})^2 + (q_2 - x_{i,2})^2} = \sqrt{(1 - x_{i,1})^2 + (1 - x_{i,2})^2}.$$

2. Cálculo de distancias a cada observación de entrenamiento

- **Para $x_1 = (0, 0)^{\top}$ (Clase A):**

$$d(q, x_1) = \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1{,}4142.$$

- **Para $x_2 = (1,0)^\top$ (Clase A):**

$$d(q, x_2) = \sqrt{(1-1)^2 + (1-0)^2} = \sqrt{0^2 + 1^2} = \sqrt{1} = 1{,}0000.$$

- **Para $x_3 = (3,1)^\top$ (Clase B):**

$$d(q, x_3) = \sqrt{(1-3)^2 + (1-1)^2} = \sqrt{(-2)^2 + 0^2} = \sqrt{4} = 2{,}0000.$$

- **Para $x_4 = (0,3)^\top$ (Clase B):**

$$d(q, x_4) = \sqrt{(1-0)^2 + (1-3)^2} = \sqrt{1^2 + (-2)^2} = \sqrt{1+4} = \sqrt{5} \approx 2{,}2361.$$

- **Para $x_5 = (1,1{,}1)^\top$ (Clase B):**

$$d(q, x_5) = \sqrt{(1-1)^2 + (1-1{,}1)^2} = \sqrt{0^2 + (-0{,}1)^2} = \sqrt{0{,}01} = 0{,}1000.$$

Orden	**Observación (x_i)**	**Coordenadas**	**Distancia $d(q, x_i)$**	**Clase real**
1	x_5	$(1,1{,}1)$	0,1000	B
2	x_2	$(1,0)$	1,0000	A
3	x_1	$(0,0)$	$\sqrt{2} \approx 1{,}4142$	A
4	x_3	$(3,1)$	2,0000	B
5	x_4	$(0,3)$	$\sqrt{5} \approx 2{,}2361$	B

Cuadro 2: Distancias euclidianas desde la consulta $q = (1,1)$ ordenadas de menor a mayor.

3. Ordenamiento de observaciones por cercanía

4. Clasificación por votación mayoritaria

- **Caso $k = 1$:**
  - Vecino más cercano: $\{x_5\}$ con $d(q, x_5) = 0{,}1000$.
  - Conteo de votos: Clase B = 1 voto, Clase A = 0 votos.
  - **Predicción:** $\hat{y}_q$ = Clase B.
- **Caso $k = 3$:**
  - Tres vecinos más cercanos: $\{x_5, x_2, x_1\}$.
  - Conteo de votos:
    - Clase A: 2 votos (x_2 y x_1).
    - Clase B: 1 voto (x_5).
  - **Predicción:** $\hat{y}_q$ = Clase A (por mayoría $2 > 1$).

5. Justificación del cambio de predicción según k El valor de k determina el tamaño de la vecindad y el grado de suavizado en la decisión:

- **Para $k = 1$ (alta varianza/sensibilidad local):** El modelo considera únicamente la observación más inmediata. Como x_5 está extremadamente cerca de q ($d = 0{,}1$), determina por completo el resultado hacia la Clase B.
- **Para $k = 3$ (mayor suavizado/promedio regional):** Al expandir la búsqueda a 3 vecinos, el algoritmo incorpora la distribución local de las clases A (x_1 y x_2) y B (x_5). La mayoría prevalece (2 contra 1), reduciendo el efecto del punto x_5 y cambiando la predicción a Clase A.

Punto 2.2: Implementación de k-NN en sintético `make_moons`

1. Configuración del experimento Se generó un conjunto de datos sintético bidimensional no linealmente separable utilizando la función `make_moons` de `scikit-learn` con los siguientes parámetros:

- Número total de observaciones: $N = 300$.
- Nivel de ruido: `noise = 0.20`.
- Semilla pseudoaleatoria: `random_state = 42`.

El conjunto se dividió en un 70 % para entrenamiento ($N_{\text{train}} = 210$) y un 30 % para prueba ($N_{\text{test}} = 90$), estratificando según la variable objetivo y para conservar la proporción equilibrada de ambas clases.

2. Resultados cuantitativos de exactitud Se entrenó el clasificador `KNeighborsClassifier` para $k \in \{1, 5, 15\}$. Los resultados de exactitud (*accuracy*) se resumen en la Tabla 3.

Valor de k	Exactitud entrenamiento (Train)	Exactitud prueba (Test)
$k = 1$	1,0000 (100,00 %)	0,9556 (95,56 %)
$k = 5$	0,9667 (96,67 %)	0,9667 (96,67 %)
$k = 15$	0,9571 (95,71 %)	0,9778 (97,78 %)

Cuadro 3: Exactitud en conjuntos de entrenamiento y prueba para distintos valores de k.

3. Código Python utilizado

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from matplotlib.colors import ListedColormap
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Generar datos y dividir 70/30 estratificado
X, y = make_moons(n_samples=300, noise=0.20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Evaluación para k in {1, 5, 15}
for k in [1, 5, 15]:
    clf = KNeighborsClassifier(n_neighbors=k)
    clf.fit(X_train, y_train)
    acc_tr = accuracy_score(y_train, clf.predict(X_train))
    acc_te = accuracy_score(y_test, clf.predict(X_test))
    print(f"k={k} | Train: {acc_tr:.4f} | Test: {acc_te:.4f}")
```

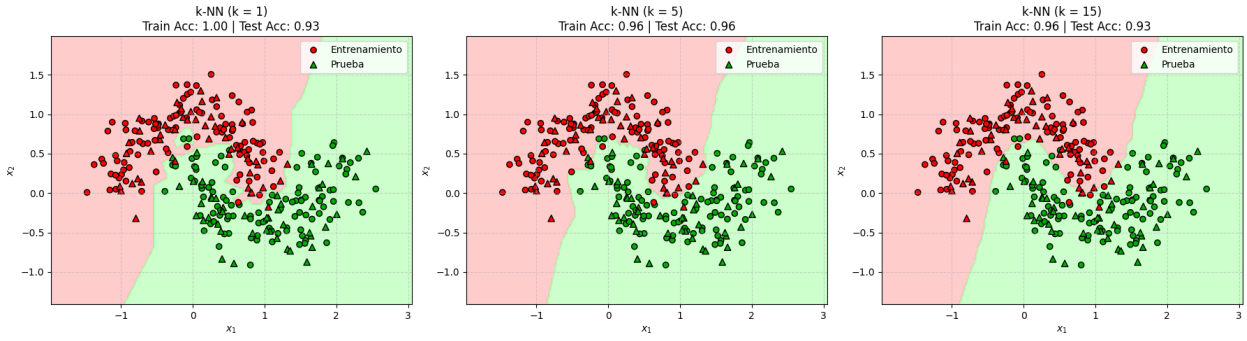


Figura 1: Fronteras de decisión de k-NN y exactitudes de entrenamiento y prueba para $k = 1$, $k = 5$ y $k = 15$.

Nota: Las exactitudes mostradas en la figura difieren de las de la tabla anterior; conviene verificar que ambas correspondan a la misma partición de los datos.

Punto 2.3: Interpretación del impacto de k y validación de modelos

1. Impacto del tamaño de k en el sesgo y la varianza El hiperparámetro k en el algoritmo k-NN controla la complejidad del modelo y el compromiso entre sesgo y varianza (*bias-variance tradeoff*):

- **k muy pequeño (por ejemplo, $k = 1$):**

- **Comportamiento:** El modelo asigna la clase basándose únicamente en el vecino más cercano, lo que puede generar fronteras de decisión muy complejas, rugosas y fragmentadas.
- **Propiedades estadísticas:** Presenta bajo sesgo y alta varianza.
- **Riesgo (*overfitting*):** Puede memorizar el conjunto de entrenamiento y alcanzar un 100 % de exactitud en *train*. Sin embargo, es susceptible al ruido y a observaciones atípicas (*outliers*), lo que puede reducir su capacidad de generalización.

- k **muy grande (por ejemplo,** $k \to N$**):**
  - **Comportamiento:** La vecindad abarca una fracción mayoritaria o la totalidad del conjunto de datos. La frontera de decisión puede volverse excesivamente suave y rígida.
  - **Propiedades estadísticas:** Presenta alto sesgo y baja varianza.
  - **Riesgo (*underfitting*):** Puede ignorar estructuras y patrones locales no lineales del espacio de características. En el límite ($k = N$), la predicción no depende de la posición de la consulta, sino de la clase mayoritaria global (o de la regla de desempate si las clases están empatadas).

2. Comparación de los resultados reportados De acuerdo con los resultados del punto 2.2, las diferencias entre las exactitudes de entrenamiento y prueba son:

- Para $k = 1$: exactitud de prueba = 95,56 % ($\text{Diff}_{\text{train-test}} = 4{,}44\,\%$).
- Para $k = 5$: exactitud de prueba = 96,67 % ($\text{Diff}_{\text{train-test}} = 0{,}00\,\%$).
- Para $k = 15$: exactitud de prueba = 97,78 % ($\text{Diff}_{\text{train-test}} = -2{,}07\,\%$).

En esta comparación, $k = 15$ obtuvo la mayor exactitud observada en el conjunto de prueba (97,78 %). Este resultado sugiere que, para esta partición, un vecindario más amplio pudo suavizar parte del ruido (`noise = 0.20`). Sin embargo, como los resultados provienen del conjunto de prueba, no se deben usar por sí solos para elegir el hiperparámetro definitivo; esa elección debe realizarse con validación y evaluarse una sola vez en prueba.

3. Justificación del uso de validación para la selección de hiperparámetros La selección formal del hiperparámetro óptimo k debe realizarse mediante un conjunto de validación o validación cruzada (*K-fold cross-validation*), y no sobre el conjunto de prueba, por los siguientes motivos:

1. **Prevención de fuga de información (*data leakage/data snooping*):** El conjunto de prueba tiene como propósito estimar el error de generalización del modelo final ante datos desconocidos. Si se utiliza para tomar decisiones de diseño, como seleccionar k, la información de prueba influye en el ajuste del modelo.
2. **Estimación fiable de la generalización:** Optimizar hiperparámetros contra el conjunto de prueba puede producir una estimación demasiado optimista del desempeño, que deja de ser una evaluación independiente de la selección realizada.

3. **Protocolo correcto:** Se dividen los datos en conjuntos independientes: *train* (para ajustar el modelo), *validation* (para seleccionar k) y *test* (para la evaluación final, una vez concluida la selección).

Punto 3.1: Matriz de confusión e identificación de componentes

1. Definición del conjunto de datos Se cuenta con un conjunto de $N = 10$ observaciones de etiquetas verdaderas y y sus correspondientes predicciones $\hat{y}$:

$$y = [0,\ 0,\ 0,\ 0,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1],$$
$$\hat{y} = [0,\ 1,\ 0,\ 0,\ 1,\ 0,\ 1,\ 1,\ 0,\ 1].$$

Tomando la clase 1 como la clase **positiva** y la clase 0 como la clase **negativa**.

2. Identificación elemento a elemento Analizamos la coincidencia entre la clase real (y_i) y la clase predicha ($\hat{y}_i$) para cada una de las 10 observaciones:

1. $y_1 = 0,\ \hat{y}_1 = 0 \implies$ Verdadero Negativo (**TN**).
2. $y_2 = 0,\ \hat{y}_2 = 1 \implies$ Falso Positivo (**FP**).
3. $y_3 = 0,\ \hat{y}_3 = 0 \implies$ Verdadero Negativo (**TN**).
4. $y_4 = 0,\ \hat{y}_4 = 0 \implies$ Verdadero Negativo (**TN**).
5. $y_5 = 1,\ \hat{y}_5 = 1 \implies$ Verdadero Positivo (**TP**).
6. $y_6 = 1,\ \hat{y}_6 = 0 \implies$ Falso Negativo (**FN**).
7. $y_7 = 1,\ \hat{y}_7 = 1 \implies$ Verdadero Positivo (**TP**).
8. $y_8 = 1,\ \hat{y}_8 = 1 \implies$ Verdadero Positivo (**TP**).
9. $y_9 = 1,\ \hat{y}_9 = 0 \implies$ Falso Negativo (**FN**).
10. $y_{10} = 1,\ \hat{y}_{10} = 1 \implies$ Verdadero Positivo (**TP**).

3. Conteos fundamentales Sumando los eventos identificados:

- **Verdaderos Positivos (TP):** Casos donde $y = 1$ y $\hat{y} = 1$: TP = **4** (índices 5, 7, 8 y 10).
- **Verdaderos Negativos (TN):** Casos donde $y = 0$ y $\hat{y} = 0$: TN = **3** (índices 1, 3 y 4).
- **Falsos Positivos (FP):** Casos donde $y = 0$ y $\hat{y} = 1$: FP = **1** (índice 2).
- **Falsos Negativos (FN):** Casos donde $y = 1$ y $\hat{y} = 0$: FN = **2** (índices 6 y 9).

Comprobación del total de muestras:

$$N = \text{TP} + \text{TN} + \text{FP} + \text{FN} = 4 + 3 + 1 + 2 = 10.$$

4. Construcción de la matriz de confusión Organizando las filas según la clase real (y) y las columnas según la clase predicha ($\hat{y}$):

		Clase predicha ($\hat{y}$)		
		0 (Negativo)	**1 (Positivo)**	**Total real**
Clase real (y)	**0 (Negativo)**	TN = 3	FP = 1	4
	1 (Positivo)	FN = 2	TP = 4	6
	Total predicho	5	5	10

Cuadro 4: Matriz de confusión para el clasificador binario evaluado.

Punto 3.2: Cálculo manual y explicación de métricas de evaluación

1. Valores base obtenidos de la matriz de confusión A partir del análisis realizado en el punto 3.1, se tienen los siguientes valores numéricos:

$$\begin{aligned} \text{TP} &= 4 \quad \text{(verdaderos positivos)}, \\ \text{TN} &= 3 \quad \text{(verdaderos negativos)}, \\ \text{FP} &= 1 \quad \text{(falsos positivos)}, \\ \text{FN} &= 2 \quad \text{(falsos negativos)}. \end{aligned}$$

2. Cálculo manual detallado de métricas

1. **Exactitud (*accuracy*):**

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{4 + 3}{4 + 3 + 1 + 2} = \frac{7}{10} = 0{,}7000 \quad (70{,}00\,\%).$$

2. **Sensibilidad/exhaustividad (*recall*):**

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{4}{4 + 2} = \frac{4}{6} = \frac{2}{3} \approx 0{,}6667 \quad (66{,}67\,\%).$$

3. **Precisión (*precision*):**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{4}{4 + 1} = \frac{4}{5} = 0{,}8000 \quad (80{,}00\,\%).$$

4. **Puntaje F_1 (F_1-*score*):** Calculando mediante la media armónica de *precision* y *recall*:

$$\begin{aligned} F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= 2 \cdot \frac{0{,}8000 \cdot \frac{2}{3}}{0{,}8000 + \frac{2}{3}} = 2 \cdot \frac{\frac{4}{5} \cdot \frac{2}{3}}{\frac{4}{5} + \frac{2}{3}} \\ &= 2 \cdot \frac{\frac{8}{15}}{\frac{22}{15}} = 2 \cdot \frac{8}{22} = \frac{16}{22} = \frac{8}{11} \approx 0{,}7273 \quad (72{,}73\,\%). \end{aligned}$$

3. Explicación conceptual de cada métrica

- ***Accuracy* (exactitud):** Evalúa la proporción de predicciones correctas respecto al total de observaciones. Indica el desempeño general del clasificador, sin distinguir entre clases.
- ***Recall* (sensibilidad/exhaustividad):** Mide la capacidad del modelo para identificar los casos que realmente pertenecen a la clase positiva. Responde a la pregunta: *De todos los elementos verdaderamente positivos, ¿qué porcentaje detectó el modelo?*
- ***Precision* (precisión):** Mide la confiabilidad de las predicciones positivas. Responde a la pregunta: *De todos los casos que el clasificador etiquetó como positivos, ¿qué porcentaje era realmente positivo?*
- F_1-*score* (puntaje F_1)**:** Es la media armónica de la precisión y la sensibilidad. Resume ambas métricas y resulta útil cuando interesa equilibrar la detección de positivos con la confiabilidad de las predicciones positivas.

Métrica	Expresión fraccionaria	Valor decimal (porcentaje)
Accuracy	7/10	0,7000 (70,00 %)
Recall	2/3	0,6667 (66,67 %)
Precision	4/5	0,8000 (80,00 %)
F_1-score	8/11	0,7273 (72,73 %)

Cuadro 5: Resultados cuantitativos de las métricas de evaluación calculadas manualmente.

Resumen de métricas

Punto 3.3: Verificación en Python y análisis crítico de la exactitud

1. Comprobación de resultados mediante `scikit-learn` Se verificaron los resultados en Python utilizando los módulos de `sklearn.metrics`. Los valores obtenidos coinciden con los cálculos manuales desarrollados en los puntos 3.1 y 3.2:

- **Matriz de confusión (`confusion_matrix`):**

$$\mathbf{C} = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \implies \text{TN} = 3,\ \text{FP} = 1,\ \text{FN} = 2,\ \text{TP} = 4.$$

- **Exactitud (`accuracy_score`):** 0,7000 (70,00 %).
- **Precisión (`precision_score`):** 0,8000 (80,00 %).
- **Sensibilidad (`recall_score`):** 0,6667 (66,67 %).
- **Puntaje F_1 (`f1_score`):** 0,7273 (72,73 %).

2. Situaciones donde la exactitud (*accuracy*) resulta engañosa La exactitud mide la proporción de aciertos globales respecto al total de datos. Aunque es intuitiva, presenta limitaciones teóricas y prácticas en los siguientes escenarios:

1. **Conjuntos de datos fuertemente desbalanceados (*class imbalance*):**
   - **Contexto:** Considérese la detección de una enfermedad rara o de fraude financiero, donde el 99 % de las transacciones son legítimas (clase 0) y solo el 1 % corresponde a fraude (clase 1).
   - **Problema:** Un clasificador trivial que siempre prediga la clase mayoritaria (clase 0) obtendrá una exactitud del **99**,**00** %.
   - **Consecuencia:** A pesar de su alta exactitud, el modelo no detecta los casos de fraude y su sensibilidad para la clase positiva es Recall = 0 %.
2. **Costos o consecuencias asimétricas del error (*asymmetric misclassification costs*):**
   - **Contexto:** En un diagnóstico oncológico, clasificar a un paciente enfermo como sano (falso negativo, FN) puede privarlo de un tratamiento oportuno. En cambio, clasificar a una persona sana como potencialmente enferma (falso positivo, FP) puede requerir exámenes confirmatorios adicionales.
   - **Problema:** La exactitud no distingue entre falsos positivos y falsos negativos; ambos cuentan como errores.
   - **Consecuencia:** Dos modelos con la misma exactitud del 90 % pueden tener perfiles de error muy distintos: uno puede cometer solo falsos positivos y el otro solo falsos negativos. En contextos donde los costos difieren, conviene considerar métricas como el *recall* o el puntaje F_β, además de los costos específicos de cada error.